EXPERIMENT 08

Configuring User Management and Role-Based Access Control (RBAC) in Zoho Creator

LOW CODE APPLICATION • LABORATORY REPORT

Student Name	Aravindh Shankar
Roll Number	241401012
Programme / Year	B.Tech Computer Science and Business Systems — III Year
Institution	Rajalakshmi Engineering College
Course	Low Code Application

Experiment aim

To design role-based access for a Creator application so that users can perform their tasks without receiving unnecessary access to personal or administrative data.

Application / case context

The Gym Enrollment app contains names, phone numbers, email addresses, residential addresses, and fitness/medical notes. The supplied form-builder screenshot marks the Member Name field as containing personal data. This makes access control important even for a small lab application.

Report structure: background, design, procedure, verification, results, conclusion and references. Any feature not confirmed by supplied evidence is identified as a proposed design or procedure.

241401012

Student: Aravindh Shankar • Roll No. 241401012 • Rajalakshmi Engineering College

1. Aim, Objectives and RBAC

Role-based access control assigns permissions to roles rather than configuring every user independently. A role represents a job function; permissions determine which forms, reports, records, and actions the role can access.

Objectives: identify user groups; apply least privilege; distinguish application administrators from front-desk staff and trainers; restrict sensitive fields and reports; and test access using separate accounts or role previews.

Proposed roles: Administrator (configure app and workflows), Front Desk (create enrollments and review contact/membership records), Trainer (view only the member information needed for training), and Read-only Auditor (review approved reports without editing records).

Permissions should be assigned according to the work a user must perform. A front-desk user who creates enrollments does not automatically need permission to modify workflows, invite administrators, or export every member record. A trainer may need a subset of member information, while an auditor may need read-only access to a limited report.

Role-based access is only effective when all access paths are considered. A restricted report may still expose data through another report, a shared link, an export, an API connection, or a page that embeds the same records. Test the permission boundary directly, including attempts to open a restricted item by its direct URL where safe and supported.

Permission reviews should be repeated whenever new forms, fields, reports, pages, or integrations are introduced. New features can expose information in ways the original role design did not anticipate. Record the reason for each role's access and remove temporary privileges once the related task is complete.

The final permission model should be understandable to a reviewer and simple enough to maintain. Avoid creating many overlapping roles without a clear difference in responsibility. Where the current application lacks record-level assignment rules, state that limitation and do not claim that the design already enforces it.

The scope of this experiment is deliberately limited to a lab-scale application. It demonstrates the underlying concept and the verification method; it does not establish production readiness without the additional checks described in this report.

Objectives and scope

This experiment studies role definitions, permission boundaries, sensitive-field protection, and user access reviews. The expected output is a role matrix and evidence that prohibited actions are denied.

2414m1013

2. Roles, Permissions and Data Classification

Member identity and contact fields are personal information. Fitness goals and medical notes may contain sensitive information and should have more restrictive visibility. A role should not receive access simply because a field is available in the form.

Create a permission matrix before configuring the application. Decide who can add, view, edit, delete, export, and share records. The permissions should align with the actual duties of each role.

Example: front-desk users may need to create and update contact and membership details. Trainers may need member name, preferred activities, and relevant fitness notes, but may not need to export the entire enrollment database. Administrators can manage configuration, but administrative access should be limited to trusted users.

Permissions should be assigned according to the work a user must perform. A front-desk user who creates enrollments does not automatically need permission to modify workflows, invite administrators, or export every member record. A trainer may need a subset of member information, while an auditor may need read-only access to a limited report.

Role-based access is only effective when all access paths are considered. A restricted report may still expose data through another report, a shared link, an export, an API connection, or a page that embeds the same records. Test the permission boundary directly, including attempts to open a restricted item by its direct URL where safe and supported.

Permission reviews should be repeated whenever new forms, fields, reports, pages, or integrations are introduced. New features can expose information in ways the original role design did not anticipate. Record the reason for each role's access and remove temporary privileges once the related task is complete.

The final permission model should be understandable to a reviewer and simple enough to maintain. Avoid creating many overlapping roles without a clear difference in responsibility. Where the current application lacks record-level assignment rules, state that limitation and do not claim that the design already enforces it.

Role testing should cover both the normal navigation path and direct access. If a user cannot see a report in the menu but can open it through a shared URL, the intended restriction has failed. Also check whether exports, embedded reports, and mobile layouts expose the same data.

Permissions should be reviewed when a user changes jobs. Removing a person from one role but leaving them in another group or sharing list can preserve access unintentionally. A simple role inventory and periodic review are more reliable than relying on memory.

An access matrix should be reviewed whenever a new report is created. The creator of a report may include fields that were previously restricted elsewhere. Review column selection, embedded report settings, export capability, and whether the report can be shared externally.

Where record-level restrictions are needed, define the rule explicitly—for example, a trainer can view only assigned members. If that rule is not supported or configured in the current app, do not claim that the trainer role is fully isolated. State the gap and describe the control that would be required.

Design / configuration summary

Role	Typical permissions	Restricted areas
Administrator	Configure forms, workflows, users, reports	Assign only trusted personnel
Front Desk	Create and update enrollments; review contact details	Avoid unnecessary access to technical settings
Trainer	View assigned members and relevant preferences	No bulk export or unrelated records by default
Read-only Auditor	View approved reports	No edits or deletes

3. Configuration Procedure

Open application Settings and locate Users, Roles, Permissions, or the corresponding access-control area in the current Creator interface. Exact labels can differ by account version and application setup.

Create or review roles, assign only the necessary form and report permissions, and add test users to the appropriate roles. Restrict edit and delete permissions for users who only need to review records.

Review sharing and portal settings separately. A portal user, application user, and administrator may have different access paths. Do not assume that hiding a navigation item alone protects the underlying data.

Permissions should be assigned according to the work a user must perform. A front-desk user who creates enrollments does not automatically need permission to modify workflows, invite administrators, or export every member record. A trainer may need a subset of member information, while an auditor may need read-only access to a limited report.

Role-based access is only effective when all access paths are considered. A restricted report may still expose data through another report, a shared link, an export, an API connection, or a page that embeds the same records. Test the permission boundary directly, including attempts to open a restricted item by its direct URL where safe and supported.

Permission reviews should be repeated whenever new forms, fields, reports, pages, or integrations are introduced. New features can expose information in ways the original role design did not anticipate. Record the reason for each role's access and remove temporary privileges once the related task is complete.

The final permission model should be understandable to a reviewer and simple enough to maintain. Avoid creating many overlapping roles without a clear difference in responsibility. Where the current application lacks record-level assignment rules, state that limitation and do not claim that the design already enforces it.

Role testing should cover both the normal navigation path and direct access. If a user cannot see a report in the menu but can open it through a shared URL, the intended restriction has failed. Also check whether exports, embedded reports, and mobile layouts expose the same data.

Permissions should be reviewed when a user changes jobs. Removing a person from one role but leaving them in another group or sharing list can preserve access unintentionally. A simple role inventory and periodic review are more reliable than relying on memory.

An access matrix should be reviewed whenever a new report is created. The creator of a report may include fields that were previously restricted elsewhere. Review column selection, embedded report settings, export capability, and whether the report can be shared externally.

Where record-level restrictions are needed, define the rule explicitly—for example, a trainer can view only assigned members. If that rule is not supported or configured in the current app, do not claim that the trainer role is fully isolated. State the gap and describe the control that would be required.

Procedure and configuration checklist

- Create a role-permission matrix before configuring users.
- Assign roles and restrict forms, reports, and sensitive fields.
- Test each role with a separate account.
- Record prohibited actions that are correctly denied.

4. Testing and Governance

Test each role with a separate account or supported permission preview. Check whether users can open the app, submit forms, view reports, edit records, delete records, export data, and access sensitive notes.

Attempt unauthorized actions directly where safe, such as opening a restricted report or record. A successful access-control test requires the action to be denied, not merely hidden from the menu.

Review user access when a person changes role or leaves the organization. Use named accounts rather than shared credentials, periodically review administrators, and document why each role needs its permissions.

Permissions should be assigned according to the work a user must perform. A front-desk user who creates enrollments does not automatically need permission to modify workflows, invite administrators, or export every member record. A trainer may need a subset of member information, while an auditor may need read-only access to a limited report.

Role-based access is only effective when all access paths are considered. A restricted report may still expose data through another report, a shared link, an export, an API connection, or a page that embeds the same records. Test the permission boundary directly, including attempts to open a restricted item by its direct URL where safe and supported.

Permission reviews should be repeated whenever new forms, fields, reports, pages, or integrations are introduced. New features can expose information in ways the original role design did not anticipate. Record the reason for each role's access and remove temporary privileges once the related task is complete.

The final permission model should be understandable to a reviewer and simple enough to maintain. Avoid creating many overlapping roles without a clear difference in responsibility. Where the current application lacks record-level assignment rules, state that limitation and do not claim that the design already enforces it.

Role testing should cover both the normal navigation path and direct access. If a user cannot see a report in the menu but can open it through a shared URL, the intended restriction has failed. Also check whether exports, embedded reports, and mobile layouts expose the same data.

Permissions should be reviewed when a user changes jobs. Removing a person from one role but leaving them in another group or sharing list can preserve access unintentionally. A simple role inventory and periodic review are more reliable than relying on memory.

An access matrix should be reviewed whenever a new report is created. The creator of a report may include fields that were previously restricted elsewhere. Review column selection, embedded report settings, export capability, and whether the report can be shared externally.

Where record-level restrictions are needed, define the rule explicitly—for example, a trainer can view only assigned members. If that rule is not supported or configured in the current app, do not claim that the trainer role is fully isolated. State the gap and describe the control that would be required.

Verification checklist

- Test allowed actions for each role.
- Attempt restricted actions directly where safe.
- Review exports, sharing and sensitive notes.

5. Results and Conclusion

The proposed RBAC model separates configuration duties from enrollment operations and trainer access. It reduces accidental disclosure and helps ensure that the minimum information necessary is visible to each role.

The supplied screenshots show an application user context and field-level personal-data marking, but do not demonstrate a complete role matrix or tests with multiple roles. Those steps must be performed in Creator before claiming RBAC is implemented.

Conclusion: RBAC is a combination of user assignment, permissions, record access, and ongoing governance. Correct configuration must be verified with test accounts and direct access checks.

Permissions should be assigned according to the work a user must perform. A front-desk user who creates enrollments does not automatically need permission to modify workflows, invite administrators, or export every member record. A trainer may need a subset of member information, while an auditor may need read-only access to a limited report.

Role-based access is only effective when all access paths are considered. A restricted report may still expose data through another report, a shared link, an export, an API connection, or a page that embeds the same records. Test the permission boundary directly, including attempts to open a restricted item by its direct URL where safe and supported.

Permission reviews should be repeated whenever new forms, fields, reports, pages, or integrations are introduced. New features can expose information in ways the original role design did not anticipate. Record the reason for each role's access and remove temporary privileges once the related task is complete.

The final permission model should be understandable to a reviewer and simple enough to maintain. Avoid creating many overlapping roles without a clear difference in responsibility. Where the current application lacks record-level assignment rules, state that limitation and do not claim that the design already enforces it.

A good conclusion separates observed results from expected results. Report any failed test and the next corrective action plainly; a documented limitation is more useful than claiming a pass that the evidence does not support.

Learning outcomes

- Designed role-based permissions.
- Applied least-privilege principles to personal data.
- Defined direct access tests and governance checks.

Conclusion

Conclusion: RBAC is a combination of user assignment, permissions, record access, and ongoing governance. Correct configuration must be verified with test accounts and direct access checks.

References

Zoho Creator Help — User management and permissions: https://www.zoho.com/creator/help/

Zoho Creator security documentation: https://www.zoho.com/creator/security.html

Zoho privacy policy: https://www.zoho.com/privacy.html

6. Role-Permission Matrix and Least Privilege

Permissions should be defined at the task level rather than assigned broadly because a user is part of the organization. Separate application administration from day-to-day data entry, and separate viewing from editing or deleting.

Record-level access can be more important than page-level access. For example, a trainer may need only members assigned to that trainer, not the complete membership database. If the current application does not implement assignment-based access, document it as a limitation instead of implying that a role name alone enforces it.

Access reviews should include active users, role membership, external sharing, portal accounts, and connections. A simple periodic review can identify old accounts or permissions that were granted temporarily and never removed.

Role testing should cover both the normal navigation path and direct access. If a user cannot see a report in the menu but can open it through a shared URL, the intended restriction has failed. Also check whether exports, embedded reports, and mobile layouts expose the same data.

Permissions should be reviewed when a user changes jobs. Removing a person from one role but leaving them in another group or sharing list can preserve access unintentionally. A simple role inventory and periodic review are more reliable than relying on memory.

Practical application to the lab

The Gym Enrollment app contains names, phone numbers, email addresses, residential addresses, and fitness/medical notes. The supplied form-builder screenshot marks the Member Name field as containing personal data. This makes access control important even for a small lab application.

Evaluation criteria

A satisfactory lab result should be reproducible, supported by a clear configuration or test record, and honest about limitations. Record the setup, test input, expected result, actual result, and evidence. If a feature was not enabled or tested in the current account, report that as a limitation and describe the procedure needed to complete it.

Checklist for submission

- Confirm the report matches the experiment title and stated objectives.
- Keep screenshots and configuration details aligned with what was actually performed.
- Separate verified outcomes from proposed implementation steps.
- Recheck the final PDF page layout before submission.

7. Access Test Plan and Final Recommendations

Use a test account for each role and verify both permitted and prohibited actions. Include opening a restricted report directly, attempting record edits, and checking export access; navigation visibility alone is insufficient.

Review permissions after adding new fields or reports because new components may expose data in ways not considered when the role was first configured. Document the intended access for every sensitive field.

Final recommendation: least privilege should be demonstrated through tests, not just described in a policy.

Role testing should cover both the normal navigation path and direct access. If a user cannot see a report in the menu but can open it through a shared URL, the intended restriction has failed. Also check whether exports, embedded reports, and mobile layouts expose the same data.

Permissions should be reviewed when a user changes jobs. Removing a person from one role but leaving them in another group or sharing list can preserve access unintentionally. A simple role inventory and periodic review are more reliable than relying on memory.

Practical application to the lab

The Gym Enrollment app contains names, phone numbers, email addresses, residential addresses, and fitness/medical notes. The supplied form-builder screenshot marks the Member Name field as containing personal data. This makes access control important even for a small lab application.

Evaluation criteria

A satisfactory lab result should be reproducible, supported by a clear configuration or test record, and honest about limitations. Record the setup, test input, expected result, actual result, and evidence. If a feature was not enabled or tested in the current account, report that as a limitation and describe the procedure needed to complete it.

Checklist for submission

- Confirm the report matches the experiment title and stated objectives.
- Keep screenshots and configuration details aligned with what was actually performed.
- Separate verified outcomes from proposed implementation steps.
- Recheck the final PDF page layout before submission.

24140101